PukSquash Developer Manual

Joining an Open Booking

Firebase Data Connect edition | Source revision fe3e361 | 4 October 2026

1 Functionality Overview

Joining adds the authenticated user as Player 2 to an existing OPEN booking. It reuses the same reservation, court, date and time slot. It must never create a second Booking for the same cell.

The availability response exposes a join action only for eligible Open cells. The browser passes the existing booking ID to joinBooking. Laravel checks the user and booking, then checks status, type, time, the user's other participation, capacity and duplicate membership. It inserts one BookingPlayer relation at position 2 and returns the original booking ID.

1 1 Files and Responsibilities

resources/js/features/booking/bookingService.js — joinBooking

resources/js/pages/BookCourtPage.vue — join action and refresh

app/Http/Controllers/Api/BookingController.php — join

app/Repositories/BookingPlayerRepository.php — add

dataconnect/example/mutations.gql — AddBookingPlayer

1 2 API Contract

POST /api/bookings/{id}/join has no user-ID payload. Success returns HTTP 200 and the existing id. Missing user/booking returns 404; invalid type, time or participation returns 422; an observed full booking returns 409 SLOT_TAKEN.

Reading this part

Code excerpts are from the stated Firebase revision unless explicitly labelled as a proposed extension. Requirements and maintenance gaps are separated from implemented behaviour. Each part can be read independently; Part 00 covers the shared environment.

2 Eligibility and capacity checks

app/Http/Controllers/Api/BookingController.php | source line 204 onward; blank and comment lines omitted

```
    if (count($booking->players) >= 2) {
        return response()->json([
            'message' => 'This booking is already full.',
            'code'    => 'SLOT_TAKEN',
        ], 409);
    }
    foreach ($booking->players as $p) {
        if (($p->user['id'] ?? null) === $user->id) {
            return response()->json(['message' => 'You are already in this booking.'],
422);
        }
    }
    $this->players->add($id, $user->id, 2);
    return response()->json(['id' => $id]);
}
```

Explanation

After verifying that the booking is confirmed and Open and that the time is still eligible, this part rejects a full booking and duplicate participation. The caller is added only when both checks pass. The response identifies the booking that was joined.

Physical Code Explanation

1 count($booking->players) >= 2 rejects an already full booking with HTTP 409.

2 The participant loop compares application user IDs, preventing the same user from occupying both positions.

3 players->add receives the existing booking ID, the verified user ID and player position 2. No CreateBooking mutation is called.

4 The final response returns the same ID so the frontend can refresh the affected booking and date.

The controller also rejects a user who already participates in any booking at that date and time slot. Treat 409 as a normal contention result: tell the user the place is unavailable and refresh the grid.

3 Persisting Player 2

dataconnect/example/mutations.gql | source line 202 onward; blank and comment lines omitted

```
mutation AddBookingPlayer(
  $bookingId: UUID!
  $userId: UUID!
  $playerPosition: Int!
  $joinedAt: Timestamp!
)
@auth(level: NO_ACCESS) {
  bookingPlayer_insert(
    data: {
      bookingId: $bookingId
      userId: $userId
      playerPosition: $playerPosition
      joinedAt: $joinedAt
    }
  )
}
```

Explanation

AddBookingPlayer inserts the relationship between the reservation and the participant. It stores the player position and join timestamp. The position is selected on the server rather than accepted from the browser.

Physical Code Explanation

1 bookingId and userId are foreign-key identifiers. playerPosition is the server-selected 1 or 2 convention.

2 joinedAt records when the participant joined. The mutation itself is a privileged server operation.

3 The standalone mutation does not read the current capacity and write the new participant atomically. The preceding controller check cannot eliminate a simultaneous Player 2 race.

Use uniqueness on the booking/position and booking/user pairs and perform eligibility plus insertion in a transaction-safe operation. Repeated or concurrent requests should return a controlled conflict without overwriting a participant.

4 Verification and Maintenance

Verification checklist

1 Join an Open booking owned by another account. Verify one new BookingPlayer at position 2 and no additional Booking row.

2 Try to join LOCKED, PRIVATE_COACH and cancelled bookings: expect rejection.

3 Try joining as the creator or an existing player. Confirm there is no second membership for the same user.

4 Try to join after the slot starts and while already booked at the same time on another court.

5 Submit two Player 2 requests concurrently. Verify one successful membership and one controlled failure after the required database protection is implemented.

Implementation boundaries

Composite uniqueness is described in schema comments but is not fully declared in the baseline BookingPlayer type. A capacity check followed by a separate insert is a known implementation gap.

When Player 2 later leaves, remove only that participant relationship. The creator’s Booking and Player 1 record continue to reserve the court.

4 1 Source References

PukSquash repository, revision fe3e361; source paths are identified beside each excerpt. Requirements: Functional Specifications PukSquash (3) and Techinical specifications PukSquash (2). Business context: Business_Case and Feasibility_Study for PukSquash. Structure reference: Campus Marketplace Developer Manual (4).